Django Real-Time Chat System - Complete Guide

This guide will walk you through building a real-time chat system with Django, Django REST Framework, and WebSockets.

Key Architectural Features:

- **Django Signals** are used for automatic profile creation and notifications
- **WebSockets** for real-time messaging
- **JWT Authentication** for secure API access
- **OTP Verification** for email validation and password reset

Table of Contents

Project Setup

Step 1: Install Required Packages

Create a new directory and set up a virtual environment:

```
# Create project directory
mkdir django_chat
cd django_chat

# Create virtual environment
python -m venv venv

# Activate virtual environment
# On Windows:
venv\Scripts\activate
# On Mac/Linux:
source venv/bin/activate

# Install required packages
pip install django djangorestframework djangorestframework-simplejwt
pip install channels channels-redis
pip install django-cors-headers
pip install python-decouple
```

Step 2: Create Django Project

```
django-admin startproject chat_project .
python manage.py startapp accounts
python manage.py startapp chat
python manage.py startapp notifications
```

Step 3: Configure settings.py

Update `chat_project/settings.py`:

```
import os
from datetime import timedelta
from pathlib import Path

BASE_DIR = Path(__file__).resolve().parent.parent

SECRET_KEY = 'your-secret-key-here-change-in-production'

DEBUG = True

ALLOWED_HOSTS = ['*']

# Application definition
INSTALLED_APPS = [
    'daphne',  # Add this at the top for channels
    'django.contrib.admin',
    'django.contrib.auth',
    'django.contrib.contenttypes',
    'django.contrib.sessions',
    'django.contrib.messages',
    'django.contrib.staticfiles',

    # Third party apps
    'rest_framework',
    'rest_framework_simplejwt',
    'corsheaders',
    'channels',

    # Local apps
    'accounts',
    'chat',
    'notifications',
]

MIDDLEWARE = [
    'django.middleware.security.SecurityMiddleware',
    'django.contrib.sessions.middleware.SessionMiddleware',
    'corsheaders.middleware.CorsMiddleware',
    'django.middleware.common.CommonMiddleware',
    'django.middleware.csrf.CsrfViewMiddleware',
    'django.contrib.auth.middleware.AuthenticationMiddleware',
```

```
    'django.contrib.messages.middleware.MessageMiddleware',
    'django.middleware.clickjacking.XFrameOptionsMiddleware',
]

ROOT_URLCONF = 'chat_project.urls'

TEMPLATES = [
    {
        'BACKEND': 'django.template.backends.django.DjangoTemplates',
        'DIRS': [],
        'APP_DIRS': True,
        'OPTIONS': {
            'context_processors': [
                'django.template.context_processors.debug',
                'django.template.context_processors.request',
                'django.contrib.auth.context_processors.auth',
                'django.contrib.messages.context_processors.messages',
            ],
        },
    },
]

WSGI_APPLICATION = 'chat_project.wsgi.application'
ASGI_APPLICATION = 'chat_project.asgi.application'

# Database
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.sqlite3',
        'NAME': BASE_DIR / 'db.sqlite3',
    }
}

# Custom User Model
AUTH_USER_MODEL = 'accounts.User'

# Password validation
AUTH_PASSWORD_VALIDATORS = [
    {'NAME':
'django.contrib.auth.password_validation.UserAttributeSimilarityValidator'},
    {'NAME': 'django.contrib.auth.password_validation.MinimumLengthValidator'},
    {'NAME': 'django.contrib.auth.password_validation.CommonPasswordValidator'},
    {'NAME': 'django.contrib.auth.password_validation.NumericPasswordValidator'},
]

# Internationalization
LANGUAGE_CODE = 'en-us'
TIME_ZONE = 'UTC'
USE_I18N = True
USE_TZ = True

# Static files
STATIC_URL = 'static/'
STATIC_ROOT = BASE_DIR / 'staticfiles'
```

```
MEDIA_URL = 'media/'
MEDIA_ROOT = BASE_DIR / 'media'

DEFAULT_AUTO_FIELD = 'django.db.models.BigAutoField'

# REST Framework
REST_FRAMEWORK = {
    'DEFAULT_AUTHENTICATION_CLASSES': (
        'rest_framework_simplejwt.authentication.JWTAuthentication',
    ),
    'DEFAULT_PERMISSION_CLASSES': [
        'rest_framework.permissions.IsAuthenticated',
    ],
}

# JWT Settings
SIMPLE_JWT = {
    'ACCESS_TOKEN_LIFETIME': timedelta(hours=1),
    'REFRESH_TOKEN_LIFETIME': timedelta(days=7),
    'ROTATE_REFRESH_TOKENS': True,
    'BLACKLIST_AFTER_ROTATION': True,
}

# CORS Settings
CORS_ALLOW_ALL_ORIGINS = True  # For development only

# Channels
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels.layers.InMemoryChannelLayer'
    }
}

# Email Configuration (using console backend for development)
EMAIL_BACKEND = 'django.core.mail.backends.console.EmailBackend'
# For production, use SMTP:
# EMAIL_BACKEND = 'django.core.mail.backends.smtp.EmailBackend'
# EMAIL_HOST = 'smtp.gmail.com'
# EMAIL_PORT = 587
# EMAIL_USE_TLS = True
# EMAIL_HOST_USER = 'your-email@gmail.com'
# EMAIL_HOST_PASSWORD = 'your-app-password'

DEFAULT_FROM_EMAIL = 'noreply@chatapp.com'
```

User Authentication System

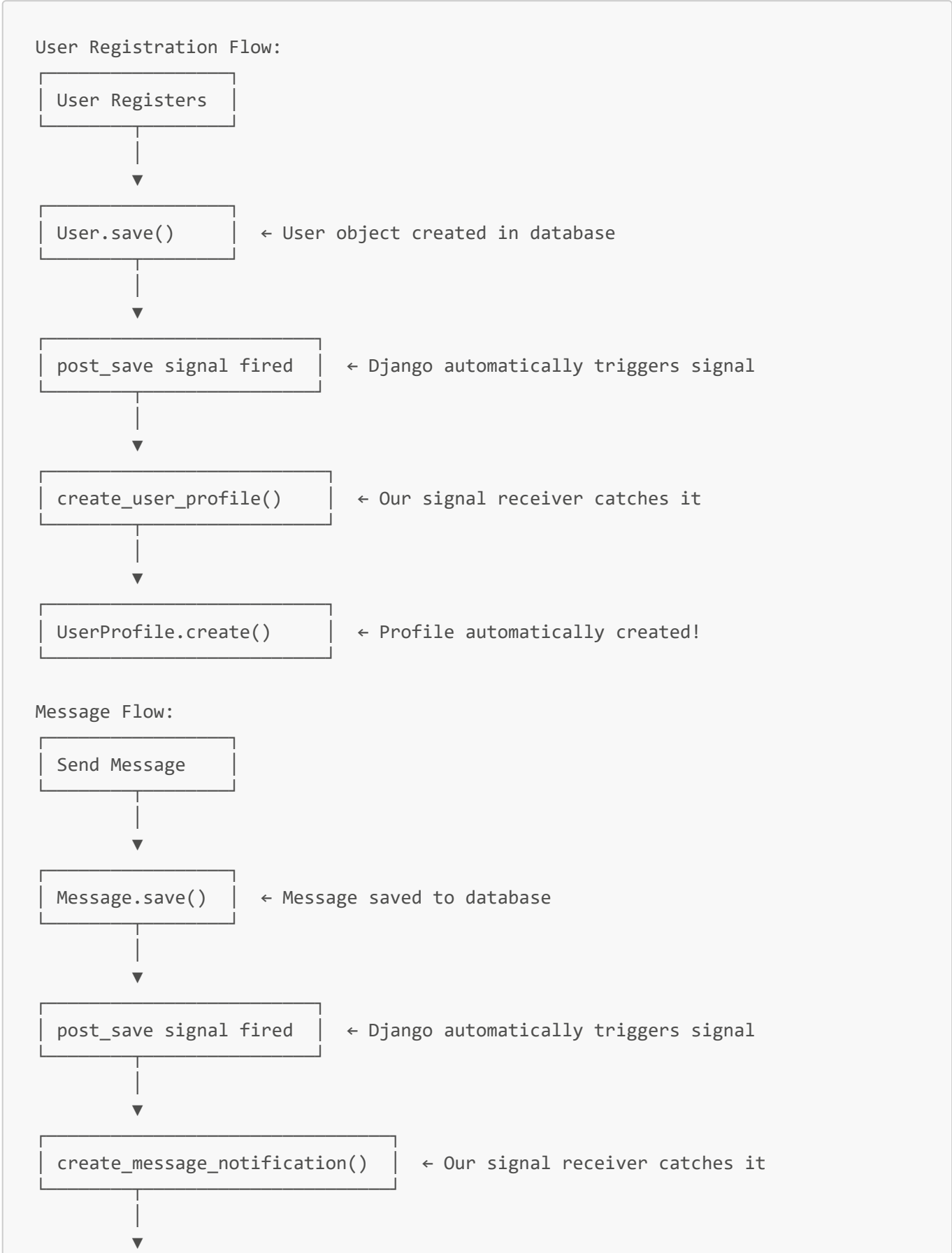
Important: Signal-based Architecture

This application uses Django signals to automatically handle related data:

1. When a `User` is created → A `UserProfile` is automatically created (via signal)
2. When a `Message` is created → A `Notification` is automatically created (via signal)

This approach keeps the code clean and follows Django best practices by separating concerns.

Signal Flow Diagram:



```
┌──────────────────────────┐
│ Notification.create()    │  ← Notification automatically created!
└──────────────────────────┘
```

Step 4: Create User Model and Profile

Create `accounts/models.py`:

```python
from django.contrib.auth.models import AbstractUser
from django.db import models
import random
from datetime import timedelta
from django.utils import timezone


class User(AbstractUser):
    email = models.EmailField(unique=True)
    is_email_verified = models.BooleanField(default=False)

    # OTP fields
    otp_code = models.CharField(max_length=6, blank=True, null=True)
    otp_purpose = models.CharField(
        max_length=20,
        blank=True,
        null=True,
        choices=[
            ('registration', 'Registration'),
            ('password_reset', 'Password Reset')
        ]
    )
    otp_created_at = models.DateTimeField(blank=True, null=True)
    otp_is_used = models.BooleanField(default=False)

    # Password Reset Token fields
    password_reset_token = models.CharField(max_length=32, blank=True, null=True)
    password_reset_token_created_at = models.DateTimeField(blank=True, null=True)
    password_reset_token_is_used = models.BooleanField(default=False)

    USERNAME_FIELD = 'email'
    REQUIRED_FIELDS = ['username']

    def generate_otp(self, purpose):
        """Generate and save OTP for the user"""
        self.otp_code = str(random.randint(100000, 999999))
        self.otp_purpose = purpose
        self.otp_created_at = timezone.now()
        self.otp_is_used = False
        self.save()
        return self.otp_code

    def verify_otp(self, code, purpose):
```

```
        """Verify OTP code"""
        if not self.otp_code or not self.otp_created_at:
            return False

        # Check if OTP matches
        if self.otp_code != code:
            return False

        # Check if purpose matches
        if self.otp_purpose != purpose:
            return False

        # Check if already used
        if self.otp_is_used:
            return False

        # Check if expired (10 minutes)
        if timezone.now() > self.otp_created_at + timedelta(minutes=2):
            return False

        return True

    def mark_otp_used(self):
        """Mark OTP as used"""
        self.otp_is_used = True
        self.save()

    def generate_password_reset_token(self):
        """Generate and save password reset token"""
        import secrets
        self.password_reset_token = secrets.token_hex(16)
        self.password_reset_token_created_at = timezone.now()
        self.password_reset_token_is_used = False
        self.save()
        return self.password_reset_token

    def verify_password_reset_token(self, token):
        """Verify password reset token"""
        if not self.password_reset_token or not
self.password_reset_token_created_at:
            return False

        # Check if token matches
        if self.password_reset_token != token:
            return False

        # Check if already used
        if self.password_reset_token_is_used:
            return False

        # Check if expired (10 minutes)
        if timezone.now() > self.password_reset_token_created_at +
timedelta(minutes=10):
            return False
```

```
        return True

    def mark_password_reset_token_used(self):
        """Mark password reset token as used"""
        self.password_reset_token_is_used = True
        self.save()

    def __str__(self):
        return self.email


class UserProfile(models.Model):
    user = models.OneToOneField(User, on_delete=models.CASCADE,
related_name='profile')
    bio = models.TextField(max_length=500, blank=True)
    avatar = models.ImageField(upload_to='avatars/', blank=True, null=True)
    phone_number = models.CharField(max_length=15, blank=True)
    date_of_birth = models.DateField(blank=True, null=True)
    location = models.CharField(max_length=100, blank=True)
    website = models.URLField(blank=True)
    created_at = models.DateTimeField(auto_now_add=True)
    updated_at = models.DateTimeField(auto_now=True)

    def __str__(self):
        return f"{self.user.email}'s Profile"
```

Create `accounts/signals.py`:

```
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.contrib.auth import get_user_model
from .models import UserProfile

User = get_user_model()


@receiver(post_save, sender=User)
def create_user_profile(sender, instance, created, **kwargs):
    """
    Signal to automatically create a UserProfile when a new User is created
    """
    if created:
        UserProfile.objects.create(user=instance)


@receiver(post_save, sender=User)
def save_user_profile(sender, instance, **kwargs):
    """
    Signal to save the UserProfile whenever the User is saved
    """
    # Only save if profile exists (it might not exist during user creation)
```

```
    if hasattr(instance, 'profile'):
        instance.profile.save()
```

Create `accounts/apps.py`:

```
from django.apps import AppConfig


class AccountsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'accounts'

    def ready(self):
        import accounts.signals  # Import signals when app is ready
```

Step 5: Create Serializers

Create `accounts/serializers.py`:

```
from rest_framework import serializers
from django.contrib.auth import get_user_model
from django.contrib.auth.password_validation import validate_password
from .models import UserProfile

User = get_user_model()


class UserProfileSerializer(serializers.ModelSerializer):
    class Meta:
        model = UserProfile
        fields = ['bio', 'avatar', 'phone_number', 'date_of_birth', 'location',
'website']


class UserSerializer(serializers.ModelSerializer):
    profile = UserProfileSerializer(read_only=True)

    class Meta:
        model = User
        fields = ['id', 'email', 'username', 'is_email_verified', 'profile']


class UserRegistrationSerializer(serializers.ModelSerializer):
    password = serializers.CharField(write_only=True, validators=
[validate_password])
    password2 = serializers.CharField(write_only=True)

    class Meta:
        model = User
```

```
        fields = ['email', 'username', 'password', 'password2']

    def validate(self, attrs):
        if attrs['password'] != attrs['password2']:
            raise serializers.ValidationError({"password": "Passwords don't
match"})
        return attrs

    def create(self, validated_data):
        validated_data.pop('password2')
        user = User.objects.create_user(
            email=validated_data['email'],
            username=validated_data['username'],
            password=validated_data['password'],
            is_active=False  # User inactive until email verified
        )
        return user


class VerifyOTPSerializer(serializers.Serializer):
    PURPOSE_CHOICES = (
        ('registration', 'Registration'),
        ('password_reset', 'Password Reset'),
    )
    email = serializers.EmailField()
    otp = serializers.CharField(max_length=6)
    purpose = serializers.ChoiceField(choices=PURPOSE_CHOICES,
default='registration')



class PasswordResetRequestSerializer(serializers.Serializer):
    email = serializers.EmailField()


class ResendOTPSerializer(serializers.Serializer):
    PURPOSE_CHOICES = (
        ('registration', 'Registration'),
        ('password_reset', 'Password Reset'),
    )
    email = serializers.EmailField()
    purpose = serializers.ChoiceField(choices=PURPOSE_CHOICES,
default='registration')


class PasswordResetConfirmSerializer(serializers.Serializer):
    email = serializers.EmailField()
    password_reset_token = serializers.CharField(max_length=32)
    new_password = serializers.CharField(write_only=True, validators=
[validate_password])
    new_password2 = serializers.CharField(write_only=True)

    def validate(self, attrs):
        if attrs['new_password'] != attrs['new_password2']:
```

```
            raise serializers.ValidationError({"new_password": "Passwords don't
match"})
        return attrs


class UserProfileUpdateSerializer(serializers.ModelSerializer):
    class Meta:
        model = UserProfile
        fields = ['bio', 'avatar', 'phone_number', 'date_of_birth', 'location',
'website']

    def update(self, instance, validated_data):
        for attr, value in validated_data.items():
            setattr(instance, attr, value)
        instance.save()
        return instance
```

Step 6: Create Views

Create `accounts/views.py`:

```
from rest_framework import status, generics
from rest_framework.response import Response
from rest_framework.permissions import AllowAny, IsAuthenticated
from django.contrib.auth import get_user_model
from django.core.mail import send_mail
from django.conf import settings
from .models import UserProfile
from .serializers import (
    UserRegistrationSerializer,
    VerifyOTPSerializer,
    PasswordResetRequestSerializer,
    ResendOTPSerializer,
    PasswordResetConfirmSerializer,
    UserSerializer,
    UserProfileUpdateSerializer
)

User = get_user_model()


class RegisterView(generics.CreateAPIView):
    queryset = User.objects.all()
    permission_classes = [AllowAny]
    serializer_class = UserRegistrationSerializer

    def create(self, request, *args, **kwargs):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        user = serializer.save()
```

```
        # Generate and save OTP
        otp_code = user.generate_otp('registration')

        # Send OTP via email
        send_mail(
            'Verify Your Email',
            f'Your OTP code is: {otp_code}\n\nThis code will expire in 2
minutes.',
            settings.DEFAULT_FROM_EMAIL,
            [user.email],
            fail_silently=False,
        )

        return Response(
            {
                "success": True,
                "status_code": 201,
                "message": "Registration successful. An OTP has been sent to your
email for verification.",
                "data": {
                    "email": user.email
                }
            },
            status=status.HTTP_201_CREATED
        )


class VerifyOTPView(generics.GenericAPIView):
    permission_classes = [AllowAny]
    serializer_class = VerifyOTPSerializer

    def post(self, request):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data['email']
        otp_code = serializer.validated_data['otp']
        purpose = serializer.validated_data.get('purpose', 'registration')

        try:
            user = User.objects.get(email=email)

            # Verify OTP with provided purpose
            if not user.verify_otp(otp_code, purpose):
                return Response({
                    'error': 'Invalid or expired OTP'
                }, status=status.HTTP_400_BAD_REQUEST)

            # Mark OTP as used
            user.mark_otp_used()

            # Update verification/activation based on purpose
            user.is_email_verified = True
            if purpose == 'registration':
```

```
                user.is_active = True
                user.save()
                return Response({
                    'message': 'Email verified successfully. You can now log in.'
                }, status=status.HTTP_200_OK)

            # For password reset, generate and return password reset token
            elif purpose == 'password_reset':
                user.save()
                password_reset_token = user.generate_password_reset_token()
                return Response({
                    'message': 'OTP verified successfully. Use the password reset
token to change your password.',
                    'password_reset_token': password_reset_token
                }, status=status.HTTP_200_OK)

            user.save()
            return Response({
                'message': 'OTP verified successfully'
            }, status=status.HTTP_200_OK)

        except User.DoesNotExist:
            return Response({
                'error': 'User not found'
            }, status=status.HTTP_404_NOT_FOUND)


class ResendOTPView(generics.GenericAPIView):
    permission_classes = [AllowAny]
    serializer_class = ResendOTPSerializer

    def post(self, request):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data['email']
        purpose = serializer.validated_data.get('purpose', 'registration')

        try:
            user = User.objects.get(email=email)

            # Generate and save new OTP
            otp_code = user.generate_otp(purpose)

            # Send OTP via email
            if purpose == 'registration':
                subject = 'Verify Your Email'
                message = f'Your OTP code is: {otp_code}\n\nThis code will expire
in 2 minutes.'
            elif purpose == 'password_reset':
                subject = 'Password Reset OTP'
                message = f'Your password reset OTP code is: {otp_code}\n\nThis
code will expire in 2 minutes.'
            else:
```

```
                subject = 'Your OTP Code'
                message = f'Your OTP code is: {otp_code}\n\nThis code will expire
in 2 minutes.'

            send_mail(
                subject,
                message,
                settings.DEFAULT_FROM_EMAIL,
                [user.email],
                fail_silently=False,
            )

            return Response({
                'message': f'OTP has been resent to your email',
                'email': user.email
            }, status=status.HTTP_200_OK)

        except User.DoesNotExist:
            # For security, don't reveal if email exists or not
            return Response({
                'message': 'If this email exists in our system, an OTP has been
resent'
            }, status=status.HTTP_200_OK)


class PasswordResetRequestView(generics.GenericAPIView):
    permission_classes = [AllowAny]
    serializer_class = PasswordResetRequestSerializer

    def post(self, request):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data['email']

        try:
            user = User.objects.get(email=email)

            # Generate and save OTP
            otp_code = user.generate_otp('password_reset')

            # Send OTP via email
            send_mail(
                'Password Reset OTP',
                f'Your password reset OTP code is: {otp_code}\n\nThis code will
expire in 2 minutes.',
                settings.DEFAULT_FROM_EMAIL,
                [user.email],
                fail_silently=False,
            )

            return Response({
                'message': 'OTP sent to your email'
            }, status=status.HTTP_200_OK)
```

```
        except User.DoesNotExist:
            # For security, don't reveal if email exists or not
            return Response({
                'message': 'If this email exists, an OTP has been sent'
            }, status=status.HTTP_200_OK)


class PasswordResetConfirmView(generics.GenericAPIView):
    permission_classes = [AllowAny]
    serializer_class = PasswordResetConfirmSerializer

    def post(self, request):
        serializer = self.get_serializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        email = serializer.validated_data['email']
        password_reset_token = serializer.validated_data['password_reset_token']
        new_password = serializer.validated_data['new_password']

        try:
            user = User.objects.get(email=email)

            # Verify password reset token
            if not user.verify_password_reset_token(password_reset_token):
                return Response({
                    'error': 'Invalid or expired password reset token'
                }, status=status.HTTP_400_BAD_REQUEST)

            # Mark password reset token as used
            user.mark_password_reset_token_used()

            # Reset password
            user.set_password(new_password)
            user.save()

            return Response({
                'message': 'Password reset successfully'
            }, status=status.HTTP_200_OK)

        except User.DoesNotExist:
            return Response({
                'error': 'User not found'
            }, status=status.HTTP_404_NOT_FOUND)


class UserListView(generics.ListAPIView):
    serializer_class = UserSerializer
    permission_classes = [IsAuthenticated]

    def get_queryset(self):
        # Return all users except the current user
        return User.objects.filter(is_active=True,
is_email_verified=True).exclude(id=self.request.user.id)
```

```
class UserProfileView(generics.RetrieveAPIView):
    serializer_class = UserSerializer
    permission_classes = [IsAuthenticated]

    def get_object(self):
        return self.request.user


class UserProfileUpdateView(generics.UpdateAPIView):
    serializer_class = UserProfileUpdateSerializer
    permission_classes = [IsAuthenticated]

    def get_object(self):
        return self.request.user.profile

    def update(self, request, *args, **kwargs):
        partial = kwargs.pop('partial', False)
        instance = self.get_object()
        serializer = self.get_serializer(instance, data=request.data,
partial=partial)
        serializer.is_valid(raise_exception=True)
        self.perform_update(serializer)

        # Return full user data with updated profile
        user_serializer = UserSerializer(request.user)
        return Response({
            'message': 'Profile updated successfully',
            'user': user_serializer.data
        })
```

Step 7: Create URLs

Create `accounts/urls.py`:

```
from django.urls import path
from rest_framework_simplejwt.views import TokenObtainPairView, TokenRefreshView
from .views import (
    RegisterView,
    VerifyEmailView,
    PasswordResetRequestView,
    PasswordResetConfirmView,
    UserListView,
    UserProfileView,
    UserProfileUpdateView
)

urlpatterns = [
    # Authentication
    path('register/', RegisterView.as_view(), name='register'),
```

```
    path('verify-otp/', VerifyOTPView.as_view(), name='verify-otp'),
    path('resend-otp/', ResendOTPView.as_view(), name='resend-otp'),
    path('login/', TokenObtainPairView.as_view(), name='login'),
    path('token/refresh/', TokenRefreshView.as_view(), name='token-refresh'),

    # Password Reset
    path('password-reset/', PasswordResetRequestView.as_view(), name='password-
reset'),
    path('password-reset/confirm/', PasswordResetConfirmView.as_view(),
name='password-reset-confirm'),

    # Users
    path('users/', UserListView.as_view(), name='user-list'),

    # Profile
    path('profile/', UserProfileView.as_view(), name='user-profile'),
    path('profile/update/', UserProfileUpdateView.as_view(), name='profile-
update'),
]
```

Admin Panel Configuration

Step 8: Register Models in Admin

Create `accounts/admin.py`:

```
from django.contrib import admin
from django.contrib.auth.admin import UserAdmin as BaseUserAdmin
from .models import User, UserProfile


class UserProfileInline(admin.StackedInline):
    model = UserProfile
    can_delete = False
    verbose_name_plural = 'Profile'
    fk_name = 'user'


class UserAdmin(BaseUserAdmin):
    inlines = (UserProfileInline,)
    list_display = ['email', 'username', 'is_email_verified', 'is_active',
'is_staff']
    list_filter = ['is_email_verified', 'is_active', 'is_staff']
    search_fields = ['email', 'username']
    ordering = ['-date_joined']

    fieldsets = (
        (None, {'fields': ('email', 'username', 'password')}),
        ('Personal Info', {'fields': ('first_name', 'last_name')}),
        ('Permissions', {'fields': ('is_active', 'is_staff', 'is_superuser',
```

```
'is_email_verified', 'groups', 'user_permissions')}),
        ('Important dates', {'fields': ('last_login', 'date_joined')}),
        ('OTP Info', {'fields': ('otp_code', 'otp_purpose', 'otp_created_at',
'otp_is_used')}),
    )

    add_fieldsets = (
        (None, {
            'classes': ('wide',),
            'fields': ('email', 'username', 'password1', 'password2'),
        }),
    )


admin.site.register(User, UserAdmin)


@admin.register(UserProfile)
class UserProfileAdmin(admin.ModelAdmin):
    list_display = ['user', 'phone_number', 'location', 'created_at']
    search_fields = ['user__email', 'user__username', 'phone_number']
    list_filter = ['created_at']
```

Chat System

Step 9: Create Chat Models

Create `chat/models.py`:

```
from django.db import models
from django.conf import settings


class Message(models.Model):
    sender = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='sent_messages'
    )
    recipient = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='received_messages'
    )
    content = models.TextField()
    timestamp = models.DateTimeField(auto_now_add=True)
    is_read = models.BooleanField(default=False)

    class Meta:
        ordering = ['timestamp']
```

```
    def __str__(self):
        return f"{self.sender} -> {self.recipient}: {self.content[:20]}"
```

Step 10: Create Chat Serializers

Create `chat/serializers.py`:

```
from rest_framework import serializers
from .models import Message
from django.contrib.auth import get_user_model

User = get_user_model()


class MessageSerializer(serializers.ModelSerializer):
    sender_email = serializers.EmailField(source='sender.email', read_only=True)
    recipient_email = serializers.EmailField(source='recipient.email',
read_only=True)

    class Meta:
        model = Message
        fields = ['id', 'sender', 'sender_email', 'recipient', 'recipient_email',
                  'content', 'timestamp', 'is_read']
        read_only_fields = ['sender', 'timestamp']


class ConversationSerializer(serializers.Serializer):
    user_id = serializers.IntegerField()
    user_email = serializers.EmailField()
    user_username = serializers.CharField()
    last_message = serializers.CharField()
    last_message_time = serializers.DateTimeField()
    unread_count = serializers.IntegerField()
```

Step 11: Create Chat Views

Create `chat/views.py`:

```
from rest_framework import generics, status
from rest_framework.response import Response
from rest_framework.views import APIView
from django.db.models import Q, Max, Count, Case, When
from django.contrib.auth import get_user_model
from .models import Message
from .serializers import MessageSerializer, ConversationSerializer

User = get_user_model()
```

```python
class SendMessageView(generics.CreateAPIView):
    serializer_class = MessageSerializer

    def create(self, request, *args, **kwargs):
        recipient_id = request.data.get('recipient')
        content = request.data.get('content')

        if not recipient_id or not content:
            return Response(
                {'error': 'Recipient and content are required'},
                status=status.HTTP_400_BAD_REQUEST
            )

        try:
            recipient = User.objects.get(id=recipient_id)
        except User.DoesNotExist:
            return Response(
                {'error': 'Recipient not found'},
                status=status.HTTP_404_NOT_FOUND
            )

        message = Message.objects.create(
            sender=request.user,
            recipient=recipient,
            content=content
        )

        serializer = self.get_serializer(message)
        return Response(serializer.data, status=status.HTTP_201_CREATED)


class ConversationListView(APIView):
    def get(self, request):
        user = request.user

        # Get all users the current user has conversations with
        conversations = Message.objects.filter(
            Q(sender=user) | Q(recipient=user)
        ).values(
            'sender', 'recipient'
        ).annotate(
            last_message_time=Max('timestamp')
        ).order_by('-last_message_time')

        conversation_list = []
        seen_users = set()

        for conv in conversations:
            other_user_id = conv['recipient'] if conv['sender'] == user.id else
conv['sender']

            if other_user_id in seen_users:
                continue
```

```
            seen_users.add(other_user_id)

            try:
                other_user = User.objects.get(id=other_user_id)
            except User.DoesNotExist:
                continue

            # Get last message
            last_message = Message.objects.filter(
                Q(sender=user, recipient=other_user) |
                Q(sender=other_user, recipient=user)
            ).order_by('-timestamp').first()

            # Count unread messages
            unread_count = Message.objects.filter(
                sender=other_user,
                recipient=user,
                is_read=False
            ).count()

            conversation_list.append({
                'user_id': other_user.id,
                'user_email': other_user.email,
                'user_username': other_user.username,
                'last_message': last_message.content if last_message else '',
                'last_message_time': last_message.timestamp if last_message else
None,
                'unread_count': unread_count
            })

        serializer = ConversationSerializer(conversation_list, many=True)
        return Response(serializer.data)


class MessageHistoryView(generics.ListAPIView):
    serializer_class = MessageSerializer

    def get_queryset(self):
        user = self.request.user
        other_user_id = self.kwargs.get('user_id')

        # Mark messages as read
        Message.objects.filter(
            sender_id=other_user_id,
            recipient=user,
            is_read=False
        ).update(is_read=True)

        return Message.objects.filter(
            Q(sender=user, recipient_id=other_user_id) |
            Q(sender_id=other_user_id, recipient=user)
        ).order_by('timestamp')
```

Step 12: Create Chat URLs

Create `chat/urls.py`:

```
from django.urls import path
from .views import SendMessageView, ConversationListView, MessageHistoryView

urlpatterns = [
    path('send/', SendMessageView.as_view(), name='send-message'),
    path('conversations/', ConversationListView.as_view(), name='conversations'),
    path('history/<int:user_id>/', MessageHistoryView.as_view(), name='message-
history'),
]
```

Notifications

Step 13: Create Notification Models

Create `notifications/models.py`:

```
from django.db import models
from django.conf import settings


class Notification(models.Model):
    NOTIFICATION_TYPES = [
        ('message', 'New Message'),
    ]

    user = models.ForeignKey(
        settings.AUTH_USER_MODEL,
        on_delete=models.CASCADE,
        related_name='notifications'
    )
    notification_type = models.CharField(max_length=20,
choices=NOTIFICATION_TYPES)
    message = models.TextField()
    is_read = models.BooleanField(default=False)
    is_seen = models.BooleanField(default=False)
    created_at = models.DateTimeField(auto_now_add=True)
    related_message = models.ForeignKey(
        'chat.Message',
        on_delete=models.CASCADE,
        null=True,
        blank=True
    )

    class Meta:
```

```
        ordering = ['-created_at']

    def __str__(self):
        return f"{self.user.email} - {self.notification_type}"
```

Step 14: Create Notification Serializers

Create `notifications/serializers.py`:

```
from rest_framework import serializers
from .models import Notification


class NotificationSerializer(serializers.ModelSerializer):
    class Meta:
        model = Notification
        fields = ['id', 'notification_type', 'message', 'is_read', 'created_at']
```

Step 15: Create Notification Views

Create `notifications/views.py`:

```
from rest_framework import generics, status
from rest_framework.response import Response
from rest_framework.views import APIView
from rest_framework.permissions import IsAuthenticated
from .models import Notification
from .serializers import NotificationSerializer


class NotificationListView(APIView):
    """
    GET: List all notifications for the authenticated user
    Automatically marks notifications as seen when fetched
    """
    permission_classes = [IsAuthenticated]

    def get(self, request):
        # Get all notifications for the user
        notifications = Notification.objects.filter(
            user=request.user
        ).order_by('-created_at')

        #Unseen notifications
        unseen_notifications = notifications.filter(is_seen=False)
        unseen_count = unseen_notifications.count()

        # Mark all unseen notifications as seen in a single query
```

```
        unseen_notifications.update(is_seen=True)

        serializer = NotificationSerializer(notifications, many=True)
        return Response({
            'notifications': serializer.data,
            'count': unseen_count
        })


class MarkNotificationReadView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request, notification_id):
        try:
            notification = Notification.objects.get(
                id=notification_id,
                user=request.user
            )
            notification.is_read = True
            notification.save()
            return Response({'message': 'Notification marked as read'})
        except Notification.DoesNotExist:
            return Response(
                {'error': 'Notification not found'},
                status=status.HTTP_404_NOT_FOUND
            )


class MarkAllNotificationsReadView(APIView):
    permission_classes = [IsAuthenticated]

    def post(self, request):
        Notification.objects.filter(
            user=request.user,
            is_read=False
        ).update(is_read=True)
        return Response({'message': 'All notifications marked as read'})
```

Step 16: Create Notification URLs

Create `notifications/urls.py`:

```
from django.urls import path
from .views import (
    NotificationListView,
    MarkNotificationReadView,
    MarkAllNotificationsReadView,

)
```

```
urlpatterns = [
    path('', NotificationListView.as_view(), name='notifications'),
    path('<int:notification_id>/read/', MarkNotificationReadView.as_view(),
name='mark-read'),
    path('mark-all-read/', MarkAllNotificationsReadView.as_view(), name='mark-all-
read'),

]
```

Step 17: Create Notification Signal

Create `notifications/signals.py`:

```
import redis
from django.db.models.signals import post_save
from django.dispatch import receiver
from django.db import transaction
from channels.layers import get_channel_layer
from asgiref.sync import async_to_sync
from chat.models import Message
from .models import Notification


@receiver(post_save, sender=Message)
def create_message_notification(sender, instance, created, **kwargs):
    """
    Signal to automatically create a notification when a new message is sent,
    and push the notification to the recipient over Channels in real time.
    """
    if created:
        # If the recipient currently has an open chat with the sender, do not
create/send a notification
        r = redis.Redis(host='127.0.0.1', port=6379, db=0, decode_responses=True)
        if r.sismember(f'open_chats:{instance.recipient.id}', instance.sender.id):
            return

        notification = Notification.objects.create(
            user=instance.recipient,
            notification_type='message',
            message=f"New message from {instance.sender.username}",
            related_message=instance
        )

        channel_layer = get_channel_layer()
        payload = {
            'type': 'notification_handler',
            'notification': {
                'id': notification.id,
                'notification_type': notification.notification_type,
                'message': notification.message,
                'is_read': notification.is_read,
```

```
                'is_seen': notification.is_seen,
                'created_at': notification.created_at.isoformat(),
                'related_message_id': instance.id,
            }
        }

        # Send after DB transaction commits to avoid race conditions
        def send_notification():
            async_to_sync(channel_layer.group_send)(
                f'chat_user_{instance.recipient.id}',
                payload
            )

        transaction.on_commit(send_notification)
```

Update `notifications/apps.py`:

```
from django.apps import AppConfig


class NotificationsConfig(AppConfig):
    default_auto_field = 'django.db.models.BigAutoField'
    name = 'notifications'

    def ready(self):
        import notifications.signals  # Import signals when app is ready
```

How it works:

- Every time a `Message` is created, Django automatically triggers the `post_save` signal
- Our signal receiver `create_message_notification` catches this signal
- A `Notification` is automatically created for the message recipient
- No need to manually create notifications in views or consumers!

WebSocket for Real-Time Chat

Step 18: Configure ASGI

Update `chat_project/asgi.py`:

```
import os
from django.core.asgi import get_asgi_application
from channels.routing import ProtocolTypeRouter, URLRouter
from channels.auth import AuthMiddlewareStack
from channels.security.websocket import AllowedHostsOriginValidator
```

```
os.environ.setdefault('DJANGO_SETTINGS_MODULE', 'chat_project.settings')

django_asgi_app = get_asgi_application()

from chat.routing import websocket_urlpatterns

application = ProtocolTypeRouter({
    "http": django_asgi_app,
    "websocket": AllowedHostsOriginValidator(
        AuthMiddlewareStack(
            URLRouter(websocket_urlpatterns)
        )
    ),
})
```

Step 19: Create WebSocket Consumer

Create `chat/consumers.py`:

```
import json
import redis
from channels.generic.websocket import AsyncWebsocketConsumer
from channels.db import database_sync_to_async
from asgiref.sync import sync_to_async
from django.contrib.auth import get_user_model
from .models import Message
from notifications.models import Notification

User = get_user_model()


class ChatConsumer(AsyncWebsocketConsumer):
    async def connect(self):
        self.user = self.scope['user']

        if self.user.is_anonymous:
            await self.close()
            return

        # Redis client for tracking open chats (per-user set)
        self.redis = redis.Redis(host='127.0.0.1', port=6379, db=0,
decode_responses=True)

        self.room_name = f'user_{self.user.id}'
        self.room_group_name = f'chat_{self.room_name}'

        # Join room group
        await self.channel_layer.group_add(
            self.room_group_name,
            self.channel_name
        )
```

```
        await self.accept()

    async def disconnect(self, close_code):
        # Leave room group
        if hasattr(self, 'room_group_name'):
            await self.channel_layer.group_discard(
                self.room_group_name,
                self.channel_name
            )

    async def receive(self, text_data):
        data = json.loads(text_data)
        message_type = data.get('type')

        if message_type == 'chat_message':
            recipient_id = data.get('recipient_id')
            content = data.get('content')

            # Save message to database
            message = await self.save_message(recipient_id, content)

            if message:
                # Send message to sender
                await self.send(text_data=json.dumps({
                    'type': 'chat_message',
                    'message': {
                        'id': message.id,
                        'sender_id': message.sender.id,
                        'sender_email': message.sender.email,
                        'recipient_id': message.recipient.id,
                        'content': message.content,
                        'timestamp': message.timestamp.isoformat(),
                    }
                }))

                # Send message to recipient
                await self.channel_layer.group_send(
                    f'chat_user_{recipient_id}',
                    {
                        'type': 'chat_message_handler',
                        'message': {
                            'id': message.id,
                            'sender_id': message.sender.id,
                            'sender_email': message.sender.email,
                            'recipient_id': message.recipient.id,
                            'content': message.content,
                            'timestamp': message.timestamp.isoformat(),
                        }
                    }
                )
        elif message_type == 'open_chat':
            other_id = data.get('chat_with')
            if other_id:
```

```
                # Add to Redis set
                await sync_to_async(self.redis.sadd)(f'open_chats:{self.user.id}',
other_id)
                # Mark related notifications as seen/read
                await self.mark_notifications_seen(other_id)
                await self.send(text_data=json.dumps({'type': 'open_chat_ack',
'chat_with': other_id}))
        elif message_type == 'close_chat':
            other_id = data.get('chat_with')
            if other_id:
                await sync_to_async(self.redis.srem)(f'open_chats:{self.user.id}',
other_id)
                await self.send(text_data=json.dumps({'type': 'close_chat_ack',
'chat_with': other_id}))

    async def chat_message_handler(self, event):
        message = event['message']

        # Send message to WebSocket
        await self.send(text_data=json.dumps({
            'type': 'chat_message',
            'message': message
        }))

    async def notification_handler(self, event):
        notification = event['notification']
        # Send notification to WebSocket
        await self.send(text_data=json.dumps({
            'type': 'notification',
            'notification': notification
        }))

    @database_sync_to_async
    def save_message(self, recipient_id, content):
        try:
            recipient = User.objects.get(id=recipient_id)
            message = Message.objects.create(
                sender=self.user,
                recipient=recipient,
                content=content
            )
            return message
        except User.DoesNotExist:
            return None

    @database_sync_to_async
    def mark_notifications_seen(self, other_id):
        Notification.objects.filter(
            user=self.user,
            related_message__sender_id=other_id,
            is_seen=False
        ).update(is_seen=True, is_read=True)
```

Step 20: Create WebSocket Routing

Create `chat/routing.py`:

```
from django.urls import path
from . import consumers

websocket_urlpatterns = [
    path('ws/chat/', consumers.ChatConsumer.as_asgi()),
]
```

Main Project URLs

Step 21: Update Main URLs

Update `chat_project/urls.py`:

```
from django.contrib import admin
from django.urls import path, include

urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/accounts/', include('accounts.urls')),
    path('api/chat/', include('chat.urls')),
    path('api/notifications/', include('notifications.urls')),
]
```

Database Setup

Step 22: Create and Apply Migrations

```
# Create migrations
python manage.py makemigrations

# Apply migrations
python manage.py migrate

# Create superuser (optional)
python manage.py createsuperuser
```

Testing the Application

Step 23: Run the Server

```
# Start local Redis (development)
# Option A: Docker (one-off)
docker run --rm -p 6379:6379 redis:7

# Option B: Docker Compose (recommended for dev)
docker compose up -d redis

# Start Django development server (simple)
python manage.py runserver
```

Step 24: API Endpoints

Here are the available endpoints:

Authentication:

- POST `/api/accounts/register/` - Register new user
- POST `/api/accounts/verify-email/` - Verify email with OTP
- POST `/api/accounts/login/` - Login (get JWT tokens)
- POST `/api/accounts/token/refresh/` - Refresh access token
- POST `/api/accounts/password-reset/` - Request password reset OTP
- POST `/api/accounts/password-reset/confirm/` - Confirm password reset with OTP
- GET `/api/accounts/users/` - Get list of all users (authenticated)

Profile:

- GET `/api/accounts/profile/` - Get current user's profile
- PUT/PATCH `/api/accounts/profile/update/` - Update user profile

Chat:

- POST `/api/chat/send/` - Send a message
- GET `/api/chat/conversations/` - Get all conversations
- GET `/api/chat/history/<user_id>/` - Get message history with a specific user

Notifications:

- GET `/api/notifications/` - Get all notifications
- GET `/api/notifications/unread-count/` - Get unread notification count
- POST `/api/notifications/<id>/read/` - Mark notification as read
- POST `/api/notifications/mark-all-read/` - Mark all notifications as read

WebSocket:

- WS `/ws/chat/` - Real-time chat connection (authenticated). For JWT auth include token in the query string: `ws://localhost:8000/ws/chat/?token=<ACCESS_TOKEN>` (or use `wss://` + a header-based token handled by custom WebSocket JWT middleware in production).

Step 25: Testing Flow

1. **Register a User:**

```
curl -X POST http://localhost:8000/api/accounts/register/ \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user1@example.com",
    "username": "user1",
    "password": "SecurePass123!",
    "password2": "SecurePass123!"
  }'
```

2. **Check console for OTP and verify email:**

```
curl -X POST http://localhost:8000/api/accounts/verify-email/ \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user1@example.com",
    "otp": "123456"
  }'
```

3. **Login:**

```
curl -X POST http://localhost:8000/api/accounts/login/ \
  -H "Content-Type: application/json" \
  -d '{
    "email": "user1@example.com",
    "password": "SecurePass123!"
  }'
```

4. **Send a Message:**

```
curl -X POST http://localhost:8000/api/chat/send/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
  -d '{
    "recipient": 2,
    "content": "Hello there!"
  }'
```

5. **Get Notifications:**

```
curl -X GET http://localhost:8000/api/notifications/ \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

6. **Get Your Profile:**

```
curl -X GET http://localhost:8000/api/accounts/profile/ \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN"
```

7. **Update Your Profile:**

```
curl -X PATCH http://localhost:8000/api/accounts/profile/update/ \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_ACCESS_TOKEN" \
  -d '{
    "bio": "Hello! I love chatting.",
    "location": "New York",
    "phone_number": "+1234567890"
  }'
```

Production Considerations

For Production Deployment:

1. **Change SECRET_KEY** in settings.py and load it from environment variables (do NOT keep it in the repo)
2. **Set DEBUG = False** and ensure `ALLOWED_HOSTS` is explicitly configured
3. **Use PostgreSQL** instead of SQLite (recommended for reliability and concurrency)

```
CHANNEL_LAYERS = {
    'default': {
        'BACKEND': 'channels_redis.core.RedisChannelLayer',
        'CONFIG': {
            'hosts': [('127.0.0.1', 6379)],
        },
    },
}
```

4. **Set up HTTPS** for WebSocket connections (wss://)
5. **Use environment variables** for sensitive data
6. **Configure CORS** properly for your frontend domain

Next Steps

Understanding Django Signals in This Project:

This project demonstrates best practices for using Django signals:

1. **Profile Creation Signal** (`accounts/signals.py`):

    - Automatically creates a UserProfile when a User registers
    - Ensures every user has a profile without manual creation
    - Keeps registration logic clean and simple

2. **Notification Signal** (`notifications/signals.py`):
    - Automatically creates notifications when messages are sent
    - Decouples notification logic from messaging logic
    - Makes the codebase more maintainable

Benefits of using signals:

- ☑ Automatic, no manual calls needed
- ☑ Keeps code organized and maintainable
- ☑ Follows Django best practices
- ☑ Easy to extend (add more signals for new features)

Future Enhancements:

1. Build a frontend application (React, Vue, or Angular)
2. Implement file/image sharing in messages
3. Add typing indicators
4. Add message reactions
5. Implement group chat functionality
6. Add message search
7. Implement message deletion
8. Add online/offline status

Common Issues and Solutions

Issue: OTP not received

- Check console output (using console email backend)
- For production, verify SMTP settings

Issue: WebSocket connection fails

- Ensure Channels is properly installed
- Check ASGI configuration
- Verify Channel Layers configuration

Issue: JWT token errors

- Ensure token is included in Authorization header
- Check token expiration
- Verify token format: `Bearer <token>`

Issue: CORS errors

- Configure CORS_ALLOWED_ORIGINS properly
- Add frontend domain to allowed origins

This completes your Django chat system! You now have a fully functional real-time chat application with user authentication, OTP verification, and notifications.